Technical Recruitment Assessment: Data & Software Engineering

Candidate Challenge: Agri-Energy Decision Support System (DSS) Pipeline

1. Overview & Project Context

You are invited to prototype the core computational engine of an agricultural **Decision Support System (DSS)** developed as part of an international clean energy research initiative.

Smallholder agricultural communities in developing regions frequently face a dual constraint: reliance on expensive, carbon-intensive diesel pumps for irrigation, and severe post-harvest crop spoilage due to the absence of electrical processing and cold-chain infrastructure.

This Decision Support System evaluates spatial farming clusters to:

1. Ingest parcel boundaries and crop classifications from public agricultural records.
2. Calculate aggregate irrigation water demand and size solar-powered pumping systems.
3. Integrate live climatological solar insolation data to model seasonal energy generation.
4. Quantify surplus electricity and identify the most profitable Productive Use of Energy (PUE) asset (such as cold storage, grain milling, or crop drying).

2. Assessment Instructions

Core Principle: Code Quality Over Task Quantity

You are **not required to complete all four tasks**. Delivering one or two complete, well-engineered, and defensively written steps is significantly better than submitting four fragile or poorly tested steps.

- **Time Budget:** Estimated at 3 to 6 hours. Please try to not spend personal time beyond this range. Once received, you have 7 days to send your answer.

- **Pragmatic Assumptions Welcomed:** If you encounter ambiguous data points, missing attributes, or a blocking obstacle, make a defensible assumption, document it clearly in code comments or markdown cells, and proceed. You will be evaluated on your judgment and transparency.

- **Deliverable Format:** Submit either a self-contained dss_pipeline.ipynb notebook or a modular dss_pipeline.py script, eventually accompanied by an environment specification (requirements.txt or pyproject.toml).

- **No Separate Written Report Needed:** A formal written report is not required. Instead, structure your notebook or script with clear docstrings, type annotations, and descriptive markdown/comment blocks explaining your architectural choices, unit conversions, and error handling.

3. Data Sources & Reference Schemas

A. Geospatial Parcel Layer: EuroCrops Portugal

- **Direct Download URL:** https://zenodo.org/records/10118572/files/PT.zip

- **Format:** Shapefile archive (ZIP file).

- **Coordinate Systems:** Geometries are distributed in ETRS89 / Portugal TM06 (EPSG:3763) or WGS 84 (EPSG:4326). You are expected to manage coordinate reference system transformations appropriately.

B. Solar Resource API: NASA POWER

You will need to retrieve monthly climatological daily solar insolation ($kWh/m^2/day$, equivalent to Peak Sun Hours) for your cluster centroid using the public NASA POWER REST API (no API key required):

- **URL:** https://power.larc.nasa.gov/api/temporal/climatology/point?parameters=ALLSKY_SFC_SW_DWN&community=AG&longitude={lon}&latitude={lat}&format=JSON

- **Key Parameter in JSON Response:** properties.parameter.ALLSKY_SFC_SW_DWN (dictionary containing 12 monthly climatological averages and the annual average).

C. Reference Table 1: Agronomic Crop Demand

Map the first two digits of the EuroCrops EC_hcat_c code to the following macro archetypes:

HCAT Prefix	Macro Category	Water Demand	Representative Crop	Fresh Yield	Farmgate Price
10	Cereals & Grains	$35 m^3/ha/day$	Maize / Wheat	$7.0 ton/ha$	$0.25/kg
20	Orchards & Fruits	$45 m^3/ha/day$	Citrus / Fruit Tree	$12.0 ton/ha$	$0.80/kg
30	Pastures & Meadows	$15 m^3/ha/day$	Fodder Grass	$4.0 ton/ha$	$0.10/kg

50	Vegetables / High-Value	$55 m^3/ha/day$	Tomato / Pepper	$25.0 ton/ha$	$0.60/kg
Other / NaN	Fallback Field Crop	$30 m^3/ha/day$	Mixed / Pulses	$5.0 ton/ha$	$0.30/kg

D. Reference Table 2: IDCOL Solar Irrigation Pump Catalog

Select the smallest pump tier whose rated motor power meets or exceeds your calculated hydraulic power requirement:

Pump Tier	Motor Power	Dedicated PV Capacity	Installed CAPEX	Max Delivery at 40 m Head
Tier 1	$3.7 kW$ $(5.0 HP)$	$5.0 kW$	$7,500	$250 m^3/day$
Tier 2	$5.5 kW$ $(7.5 HP)$	$7.5 kW$	$10,500	$400 m^3/day$

Tier 3	$7.5kW\ (10.0HP)$	$10.0kW$	$14,000	$580m^3/day$
Tier 4	$11.0kW\ (15.0HP)$	$15.0kW$	$19,500	$850m^3/day$

E. Reference Table 3: Productive Use of Energy (PUE) Catalog

PUE Asset	Compatible HCAT	Daily Load	Installed CAPEX	Economic Value Uplift Formula
A. Cold Room (5 MT)	20, 50	$20.0kWh/day$	$6,000	Prevents 20% spoilage on total harvested volume
B. Grain Milling Unit	10	$25.0kWh/day$	$4,500	Earns $0.05/kg processing fee on milled grain
C. Solar Hybrid Dryer	All (10, 20, 50)	$10.0kWh/day$	$2,500	Yields $0.12/kg premium on 15% of harvest

4. Technical Tasks & Implementation Guidance

Task 1: Spatial Ingestion & Clustering

1. Load or stream PT.zip into a GeoPandas GeoDataFrame.

2. Accept a user-specified coordinate pair (latitude, longitude) in decimal degrees.

3. Transform coordinates into a metric planar CRS (EPSG:3763), construct a **150-meter radius circular buffer** (representing a communal tube-well command zone of approximately $7 hectares$), and identify all intersecting agricultural parcels.

Defensive Boundary Condition & Custom Exception:
If a user supplies coordinates situated in an urban center, a water body, or outside Portugal where **zero agricultural parcels intersect the 150m buffer**, your pipeline can explicitly raise a custom exception:

```
class EmptyClusterError(Exception):
    pass
```

With the explicit error message:
"Location is out of agricultural domain or contains no parcels within the 150m radius."

4. **Verification in Submission:**
   Demonstrate that your function succeeds on a valid agricultural location (e.g., Alentejo farming basin: Latitude 38.015, Longitude -7.865) and cleanly raises EmptyClusterError on an urban or invalid location (e.g., central Lisbon: Latitude 38.722, Longitude -9.139).

5. Calculate parcel surface areas in hectares and assign each parcel to an agronomic category using the first two digits of EC_hcat_c.

Key Tips for Task 1:

- **Metric Reprojection:** Never buffer directly in EPSG:4326 decimal degrees (a buffer of 150 in degrees spans thousands of kilometers). Create your point in EPSG:4326, reproject it to EPSG:3763 (meters) via .to_crs(epsg=3763), and only then apply .buffer(150).

- **Spatial Indexing Performance:** Because the national layer contains tens of thousands of polygons, avoid brute-force spatial checks. Leverage GeoPandas spatial indexing (sindex or bounding box envelope pre-filtering) to keep cluster extraction under a few seconds.

Task 2: Hydraulic Demand Sizing & NASA Climate Integration

1. Calculate the cluster aggregate peak daily volumetric water demand:

$$Q_{day} = \sum(Area_i \times WaterDemand_i)[m^3/day]$$

2. Query the NASA POWER API using the cluster centroid coordinates to extract:
    - Design month Peak Sun Hours (minimum operational monthly PSH).
    - Annual average Peak Sun Hours (PSH_{avg} in $kWh/m^2/day$).

3. Calculate required hydraulic pump power (kW) assuming:
    - Total Dynamic Head: $H = 40m$ (static water depth + friction losses).
    - Pump and motor subsystem efficiency: $\eta = 0.55$.
    - Operating pumping duration: $t_{pump} = 6hours/day$.

$$HydraulicPower(kW) = \frac{\rho \cdot g \cdot Q_{day} \cdot H}{3.6 \times 10^6 \cdot t_{pump} \cdot \eta}$$

with $\rho = 1000kg/m^3$ *and* $g = 9.81m/s^2$.

Key Tips for Task 2:

- **API Error Resilience & Caching:** External HTTP requests can fail or experience latency. Wrap your network call in a try/except block with a timeout (e.g., timeout=10), and handle JSON parsing safely. Consider a simple in-memory cache to keep notebook re-runs fast.

- **Dimensional Analysis:** In the hydraulic formula, 3.6×10^6 converts Watt-hours to Joules ($3600s \times 1000W$). Ensure flow rate and pumping duration match units cleanly.

Task 3: Equipment Matching & Surplus Energy Modeling

1. Match the required hydraulic power to the appropriate tier in the **IDCOL Pump Catalog** (Table 2). If Q_{day} exceeds Tier 4 capacity, note that multiple pumps are required.

2. Calculate average daily solar PV generation assuming a system derate factor of 0.75:

$$E_{PV} = PV_{kW} \times PSH_{avg} \times 0.75[kWh/day]$$

3. Compute daily electrical energy consumed by the pump motor:

$$E_{pump} = P_{motor} \times t_{pump}[kWh/day]$$

4. Compute net daily surplus solar energy:

$$E_{excess} = \max(0, E_{PV} - E_{pump})[kWh/day]$$

Key Tips for Task 3:

- **Programmatic Filtering:** Rather than writing complex nested if/elif/else statements, store Table 2 in a small DataFrame or list of dictionaries and filter dynamically with Pandas or Python standard library operations.

- **Power (kW) vs. Energy (kWh):** Maintain clear distinctions in naming. P_{motor} and PV_{kW} represent instantaneous capacity (kW / kW_p), while E_{PV}, E_{pump}, and E_{excess} represent cumulative daily work (kWh/day).

Task 4: Productive Use (PUE) Selection & Financial Return

1. Filter candidate PUE assets from Table 3 that satisfy both requirements:
    - **Energy Feasibility:** $E_{excess} \geq PUEDailyLoad$.

    - **Crop Relevance:** The cluster produces crops compatible with the asset's HCAT category.

2. If multiple assets qualify, select the option yielding the highest **Annual Net Financial Uplift ($/year)**.

3. Calculate the **Simple Payback Period** for the selected asset:

4. $$Payback(years) = \frac{CAPEX_{PUE}}{AnnualValueUpliftGenerated}$$

Key Tips for Task 4:

- **Volume-Dependent Uplift Logic:**
    - *Cold Room:* Prevents 20% spoilage on total harvested fresh weight ($Area \times Yield$) multiplied by farmgate price. Convert tons to kilograms ($1ton = 1000kg$) before multiplying by price per kg.
    - *Grain Milling:* Applies the $0.05/kg processing fee on milled grain harvest.
    - *Dryer:* Applies the $0.12/kg premium on 15% of total harvest volume across all compatible crops.
- **Edge Case Defensive Handling:** If no PUE asset is technically feasible (e.g., surplus energy is too low or no compatible crops are grown), handle this cleanly: return a structured dictionary or status flag indicating no investment rather than allowing a division-by-zero error on payback.

5. Submission Checklist

Please deliver your submission via email or Git repository link containing:

1. dss_pipeline.ipynb (or dss_pipeline.py with an execution runner script).
2. Environment specification (requirements.txt or pyproject.toml).
3. Inline docstrings, type annotations, and descriptive markdown blocks covering:
    - Coordinate reference systems, spatial indexing, and dimensional units.
    - Defensible engineering assumptions made to resolve blockers.
    - A concise note on how this single-cluster engine could be wrapped in a Monte Carlo loop to model regional policy incentives.

6. Evaluation Rubric

Submissions are scored on software modularity, defensive engineering, and mathematical rigor rather than raw completion percentage. Completing two or three tasks thoroughly with tests and clean error handling receives a higher score than a fragile implementation of all four steps.

Evaluation Dimension	Weight	Scoring Benchmarks
Code Structure & Modularity	**30%**	Reusable, decoupled functions with explicit type annotations and docstrings. Clean separation of concerns (ETL, API, sizing, economics). Avoids unindexed loops and monolithic blocks.
Geospatial & Dimensional Rigor	**25%**	Correct reprojection to metric planar CRS (EPSG:3763) before applying the 150m buffer. Rigorous unit handling (m^3/day, kW, kW_p, kWh/day).
Defensive Engineering & Error Handling	**25%**	Custom EmptyClusterError implemented and verified on valid vs. invalid coordinates. Resilient HTTP request handling. Clean handling of zero-excess or zero-compatible PUE cases.

Reasoning & Justified Assumptions	**20%**	Clear, transparent documentation of engineering trade-offs. Defensible and explicit assumptions when resolving missing data or simplifying real-world dynamics.